customer block is the block that will be referenced next. This assumption set is the exact opposite of the one that forms the basis for the LRU strategy. Indeed, the optimal strategy for block replacement is the **most recently used** (**MRU**) strategy. If a *customer* block must be removed from the buffer, the MRU strategy chooses the most recently used block.

For the MRU strategy to work correctly for our example, the system must pin the *customer* block currently being processed. After the final *customer* tuple has been processed, the block is unpinned, and it becomes the most recently used block.

In addition to using knowledge that the system may have about the request being processed, the buffer manager can use statistical information about the probability that a request will reference a particular relation. For example, the data dictionary that (as we will see in detail in Section 11.8) keeps track of the logical schema of the relations as well as their physical storage information is one of the most frequently accessed parts of the database. Thus, the buffer manager should try not to remove data-dictionary blocks from main memory, unless other factors dictate that it do so. In Chapter 12, we discuss indices for files. Since an index for a file may be accessed more frequently than the file itself, the buffer manager should, in general, not remove index blocks from main memory if alternatives are available.

The ideal database block-replacement strategy needs knowledge of the database operations—both those being performed and those that will be performed in the future. No single strategy is known that handles all the possible scenarios well. Indeed, a surprisingly large number of database systems use LRU, despite that strategy's faults. The exercises explore alternative strategies.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again. If the system is processing requests by several users concurrently, the concurrency-control subsystem (Chapter 16) may need to delay certain requests, to ensure preservation of database consistency. If the buffer manager is given information from the concurrency-control subsystem indicating which requests are being delayed, it can use this information to alter its block-replacement strategy. Specifically, blocks needed by active (nondelayed) requests can be retained in the buffer at the expense of blocks needed by the delayed requests.

The crash-recovery subsystem (Chapter 17) imposes stringent constraints on block replacement. If a block has been modified, the buffer manager is not allowed to write back the new version of the block in the buffer to disk, since that would destroy the old version. Instead, the block manager must seek permission from the crash-recovery subsystem before writing out a block. The crash-recovery subsystem may demand that certain other blocks be force-output before it grants permission to the buffer manager to output the block requested. In Chapter 17, we define precisely the interaction between the buffer manager and the crash-recovery subsystem.

11.6 File Organization

A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

Figure 11.6 File containing *account* records.

assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Although blocks are of a fixed size determined by the physical properties of the disk and by the operating system, record sizes vary. In a relational database, tuples of distinct relations are generally of different sizes.

One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records.

11.6.1 Fixed-Length Records

As an example, let us consider a file of *account* records for our bank database. Each record of this file is defined as:

```
type deposit = record
                   account-number : char(10);
                   branch-name : char (22);
                   balance : real;
               end
```

If we assume that each character occupies 1 byte and that a real occupies 8 bytes, our *account* record is 40 bytes long. A simple approach is to use the first 40 bytes for the first record, the next 40 bytes for the second record, and so on (Figure 11.6). However, there are two problems with this simple approach:

1. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

Figure 11.7 File of Figure 11.6, with record 2 deleted and all records moved.

2. Unless the block size happens to be a multiple of 40 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.

When a record is deleted, we could move the record that came after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 11.7). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 11.8).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record, and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need to store there is the address of the first record whose contents are deleted. We use this

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 8	A-218	Perryridge	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600

Figure 11.8 File of Figure 11.6, with record 2 deleted and final record moved.

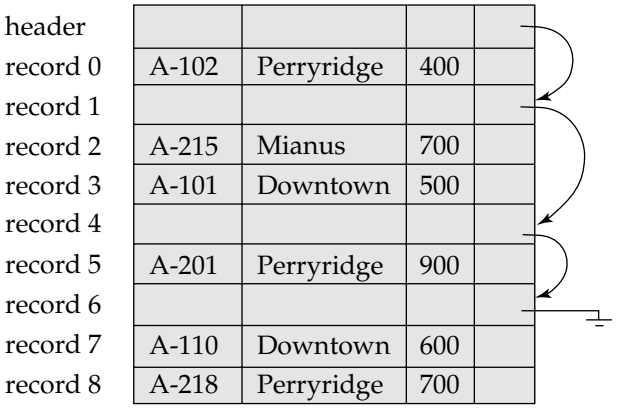


Figure 11.9 File of Figure 11.6, with free list after deletion of records 1, 4, and 6.

first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 11.9 shows the file of Figure 11.6, with the free list, after records 1, 4, and 6 have been deleted.

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file.

Insertion and deletion for files of fixed-length records are simple to implement, because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

11.6.2 Variable-Length Records

Variable-length records arise in database systems in several ways:

- Storage of multiple record types in a file
- Record types that allow variable lengths for one or more fields
- Record types that allow repeating fields

Different techniques for implementing variable-length records exist. For purposes of illustration, we shall use one example to demonstrate the various implementation techniques. We shall consider a different representation of the *account* information stored in the file of Figure 11.6, in which we use one variable-length record for each branch name and for all the account information for that branch. The format of the record is

```
type account-list = record
                        branch-name : char (22);
                        account-info : array [1 .. ∞] of
                                           record;
                                               account-number : char(10);
                                               balance : real;
                                           end
                    end
```

We define *account-info* as an array with an arbitrary number of elements. That is, the type definition does not limit the number of elements in the array, although any actual record will have a specific number of elements in its array. There is no limit on how large a record can be (up to, of course, the size of the disk storage!).

11.6.2.1 Byte-String Representation

A simple method for implementing variable-length records is to attach a special *end-of-record* (⊥) symbol to the end of each record. We can then store each record as a string of consecutive bytes. Figure 11.10 shows such an organization to represent the file of fixed-length records of Figure 11.6 as variable-length records. An alternative version of the byte-string representation stores the record length at the beginning of each record, instead of using end-of-record symbols.

The byte-string representation as described in Figure 11.10 has some disadvantages:

- It is not easy to reuse space occupied formerly by a deleted record. Although techniques exist to manage insertion and deletion, they lead to a large number of small fragments of disk storage that are wasted.
- There is no space, in general, for records to grow longer. If a variable-length record becomes longer, it must be moved—movement is costly if pointers to the record are stored elsewhere in the database (e.g., in indices, or in other records), since the pointers must be located and updated.

Thus, the basic byte-string representation described here not usually used for implementing variable-length records. However, a modified form of the byte-string repre-

0	Perryridge	A-102	400	A-201	900	A-218	700	⊥
1	Round Hill	A-305	350	⊥				
2	Mianus	A-215	700	⊥				
3	Downtown	A-101	500	A-110	600	⊥		
4	Redwood	A-222	700	⊥				
5	Brighton	A-217	750	⊥				

Figure 11.10 Byte-string representation of variable-length records.

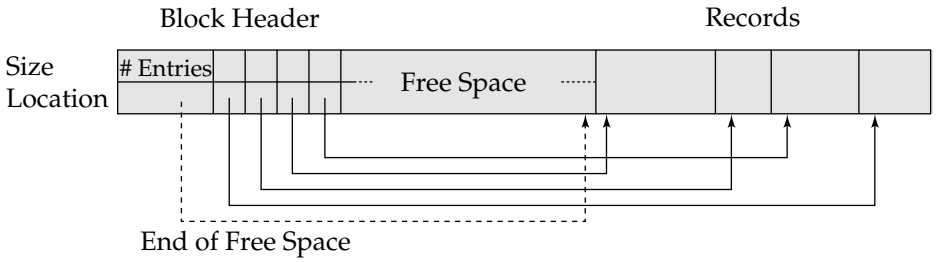


Figure 11.11 Slotted-page structure.

sentation, called the slotted-page structure, is commonly used for organizing records *within* a single block.

The **slotted-page structure** appears in Figure 11.11. There is a header at the beginning of each block, containing the following information:

1. The number of record entries in the header
2. The end of free space in the block
3. An array whose entries contain the location and size of each record

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous, between the final entry in the header array, and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to deleted (its size is set to -1, for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: A typical value is 4 kilobytes.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

11.6.2.2 Fixed-Length Representation

Another way to implement variable-length records efficiently in a file system is to use one or more fixed-length records to represent one variable-length record.

There are two ways of doing this:

1. **Reserved space**. If there is a maximum record length that is never exceeded, we can use fixed-length records of that length. Unused space (for records

0	Perryridge	A-102	400	A-201	900	A-218	700
1	Round Hill	A-305	350	⊥	⊥	⊥	⊥
2	Mianus	A-215	700	⊥	⊥	⊥	⊥
3	Downtown	A-101	500	A-110	600	⊥	⊥
4	Redwood	A-222	700	⊥	⊥	⊥	⊥
5	Brighton	A-217	750	⊥	⊥	⊥	⊥

Figure 11.12 File of Figure 11.10, using the reserved-space method.

shorter than the maximum space) is filled with a special null, or end-of-record, symbol.

2. **List representation**. We can represent variable-length records by lists of fixed-length records, chained together by pointers.

If we choose to apply the reserved-space method to our account example, we need to select a maximum record length. Figure 11.12 shows how the file of Figure 11.10 would be represented if we allowed a maximum of three accounts per branch. A record in this file is of the *account-list* type, but with the array containing exactly three elements. Those branches with fewer than three accounts (for example, Round Hill) have records with null fields. We use the symbol ⊥ to represent this situation in Figure 11.12. In practice, a particular value that can never represent real data is used (for example, an account number that is blank, or a name beginning with "*").

The reserved-space method is useful when most records have a length close to the maximum. Otherwise, a significant amount of space may be wasted. In our bank example, some branches may have many more accounts than others. This situation leads us to consider the linked list method. To represent the file by the linked list method, we add a pointer field as we did in Figure 11.9. The resulting structure appears in Figure 11.13.



Figure 11.13 File of Figure 11.10 using linked lists.